Student Number _____

Student Name _____

MOCK MIDTERM EXAMINATION

Fall 2026

CSC3002 C/C++ Programming

Practice paper based on the 26 October tutorial.

This is a mock examination, not an official university paper.

Suggested practice duration: 100 minutes. Total: 100 points.

This paper has 14 questions divided into 3 parts.

Part I: Single-choice questions (Q1-Q4), 20 points.

Part II: Multiple-choice questions (Q5-Q11), 40 points.

Part III: Written-response questions (Q12-Q14), 40 points.

Instructions

Choose one answer for each question in Part I. Select all true options in Part II. Record your choices on page 8 or beside each question.

Write Part III answers in the boxes provided. Use additional paper if necessary and label the question number.

Assume C++17, all required standard headers, and using namespace std. Snippets and options are independent unless stated otherwise. Assume allocations succeed and the small arithmetic values do not overflow.

No numerical memory addresses are requested.

Practice marking: Part II uses exact-set marking (all correct choices and no incorrect ones). The actual exam marking policy and duration were not supplied.

Part I: Single-choice questions (Q1-Q4). 20 points.

Select ONE answer for each question. Each question carries 5 points.

1. Caller variables [5 points]

```
void adjust(int a, int& b, int* c) {
    a += 2;
    b += a;
    *c += b;
}
int x=3, y=4, z=5;
adjust(x,y,&z);
cout << x << ' ' << y << ' ' << z;
```

What is printed?

A. 5 9 14

B. 3 9 14

C. 3 7 12

D. 3 4 5

Answer: _____

2. Overloading and default arguments [5 points]

```
int pick(int x) { return x+1; }
double pick(double x, int extra=2) { return x+extra; }
cout << pick(4) << ' ' << pick(4.5) << ' ' << pick(4.5,1);
```

What is printed?

A. 6 6.5 5.5

B. 5 5.5 5.5

C. 5 6.5 5.5

D. The first call is ambiguous because the second overload has a default.

Answer: _____

Part I: Single-choice questions (continued)

3. Short-circuit evaluation [5 points]

```
int visits=1;
bool a=true || (++visits > 2);
bool b=false || (++visits > 2);
bool c=b && (++visits > 0);
cout << visits << ' ' << a << ' ' << b << ' ' << c;
```

What is printed? boolalpha has not been enabled.

A. 3 1 0 1

B. 2 1 1 1

C. 4 1 1 1

D. 2 1 0 0

Answer: _____

4. Templates and division [5 points]

```
template<typename T>
T half(T v) { return v/2; }
auto a=half(9);
auto b=half(9.0);
double c=half(9);
```

Which describes a, b and c correctly?

A. a: int 4; b: double 4.5; c: double 4.0

B. a: double 4.5; b: double 4.5; c: double 4.5

C. a: int 4; b: double 4.5; c: double 4.5

D. a: int 4; b: int 4; c: double 4.0

Answer: _____

Part II: Multiple-choice questions (Q5-Q11). 40 points.

Select ALL true options. Q5-Q9 carry 6 points each; Q10-Q11 carry 5 points each.

5. Arrays, pointer parameters and const [6 points]

```
void inspect(int a[6]);  // consider the type of its parameter
int values[4]={2,4,6,8};
int* p=values;
const int* read=values;
int* const fixed=values;
```

A. inspect's parameter is int*; the 6 does not enforce the argument's array length.

B. values+4 may be formed, but reading *(values+4) is undefined behavior.

C. ++read and *fixed=9 are permitted operations, considered independently from the initial state.

D. ++fixed and *read=9 are both permitted operations.

E. Inside inspect, sizeof(a) gives the size of six int objects.

F. values=p is a valid way to reseat the array.

Answer: _____

6. Characters and strings [6 points]

```
char label[]="Go";
string word="Blue";
```

A. sizeof(label) is 3, while strlen(label) is 2.

B. word.substr(1,2) returns "lu" without changing word.

C. The expression "C" + "++" concatenates the two literals.

D. Both string("C")+"++" and "C"+string("++") produce a string containing C++.

E. strcpy(label,"Blue") is safe because strcpy resizes the destination.

F. strcmp("Go","Go") is zero.

Answer: _____

Part II: Multiple-choice questions (continued)

7. Const members and shared state [6 points]

```
struct Ticket {
    const int code;
    static int created;
    Ticket(int c) : code(c) { ++created; }
    int getCode() const { return code; }
};
int Ticket::created=0;
// In main:
Ticket a(17);
const Ticket b(42);
```

A. Ticket::created is 2 after both constructions.

B. Both a.getCode() and b.getCode() are valid calls.

C. Replacing the constructor with Ticket(int c) { code=c; ++created; } is valid.

D. Each Ticket has an independent copy of created.

E. The trailing const on getCode prevents ordinary modification of code through this; it does not restrict calls to const objects only.

F. A static member function has a this pointer for the object used to call it.

Answer: _____

Part II: Multiple-choice questions (continued)

8. Abstract classes and virtual functions [6 points]

```
struct Reporter {
    virtual int report() const=0;
    int tag() const { return 1; }
    virtual ~Reporter()=default;
};
struct Brief : Reporter {
    int report() const override { return 7; }
    int tag() const { return 2; }
};
Brief b;
Reporter& r=b;
```

A. Reporter x; is valid because Reporter has a destructor.

B. r.report() returns 7.

C. r.tag() returns 1.

D. Changing Brief::report to int report() override causes a compilation error.

E. Reporter* p=new Brief; delete p; safely performs polymorphic destruction.

F. Private members declared in Reporter would automatically be directly accessible inside Brief.

Answer: _____

Part II: Multiple-choice questions (continued)

9. Dynamic memory [6 points]

Consider each option independently.

A. int* p=new int[1]; must be paired with delete[] p; despite the length being one.

B. After int* p=new int(3); int* q=p; delete p; p=nullptr;, q still dangles.

C. After auto s=make_shared<int>(3); weak_ptr<int> w=s;, s.use_count() is 2.

D. unique_ptr<int> can be copied to create two independent owners of the same allocation.

E. For rows separately allocated using new int[cols], deleting only the outer int* array leaves the row allocations unreleased if no other owner frees them.

F. Compiler-generated copying of a class with an owning raw pointer automatically makes a deep copy of its pointee.

Answer: _____

10. Streams [5 points]

A. ostringstream builds text in memory; ofstream writes to a file.

B. Calling oss.str() alone prints its contents and clears oss.

C. oss << fixed << setprecision(2) << 3.14159; writes "3.14" into a fresh oss.

D. clear() removes the unread character that caused numeric extraction to fail.

E. while(in >> n) processes n only after successful extraction, unlike a general loop that tests !in.eof() before reading.

Answer: _____

Part II: Multiple-choice questions (continued)

11. Templates and iterators [5 points]

```
template<typename T>
T larger(T a,T b) { return a>b ? a:b; }
vector<int> v{1,2,3};
```

A. larger(2,2.5) deduces T as double automatically.

B. larger<double>(2,2.5) is valid and returns 2.5.

C. Reading *v.end() is a valid way to access the last element.

D. for(auto x:v) ++x; leaves v unchanged.

E. In an independent run, for(auto& x:v) ++x; changes v to {2,3,4}.

Answer: _____

Answer record - Parts I and II

Q1		Q2		Q3		Q4	
Q5		Q6		Q7		Q8	
Q9		Q10		Q11			

Part III: Written-response questions (Q12-Q14). 40 points.

Write your answers in the boxes. Show reasoning where requested.

12. Trace, write and diagnose. (12 points)

(a) Pointer and element state [6 points]

```
int data[5]={3,6,9,12,15};
int* p=data;
int a=*++p;
int b=*p++;
int c=++*p;
int d=(*p)++;
cout << a << ' ' << b << ' ' << c << ' ' << d << '\n';
cout << (p-data) << ' ' << *p << '\n';
```

Write both output lines [4]. State the final array and identify which of the four assignments move p [2]. Use indices, not numerical addresses.

Answer to Q12(a): output, final array and pointer movement

12. Trace, write and diagnose (continued)

(b) Function-pointer callback [4 points]

Write isEven(long), returning whether its argument is even. Then write:

```
int countIf(const long* a, int n, bool (*pred)(long));
```

It must count elements satisfying pred. Assume n>=0, a refers to at least n valid elements when n>0, and pred is valid. Finally write an explicitly typed function-pointer variable named test bound to isEven, and state the result of countIf for {3,4,8,9}, n=4 and test. Do not replace the callback with a hard-coded evenness check inside countIf.

Answer to Q12(b): function definitions, pointer declaration and result

(c) Lifetime error [2 points]

```
int* bad() { int local=7; return &local; }
int* q=bad();
cout << *q;
```

Is this necessarily a compilation error, a guaranteed output, or undefined behavior when dereferenced? Explain why.

Answer to Q12(c)

13. Operators and abstract interfaces. (14 points)

(a) Implement operators [7 points]

Complete these members inside Money. Use const-reference right operands and make + callable on a const Money. operator+ must return a new sum without changing either operand [4]. operator+= must update the left operand and support chaining [3]. Assume sums fit in int.

```
struct Money {
    int cents;
    // Write operator+ here.
    // Write operator+= here.
};
```

Answer to Q13(a): operator implementations

(b) Explain const and returned values [2 points]

Explain the distinct meanings of the const in const Money& rhs and the trailing const on operator+. Why must the proposed += not use trailing const when it changes cents?

Answer to Q13(b)

13. Operators and abstract interfaces (continued)

(c) Abstract interface [3 points]

Explain why a programmer might use the following abstract interface. Write a concrete SilentReporter whose report returns 0. State why Reporter itself cannot be instantiated.

```
struct Reporter {
    virtual int report() const=0;
    virtual ~Reporter()=default;
};
```

Answer to Q13(c): derived class and explanation

(d) Dispatch [2 points]

Using your SilentReporter, what does this print, and why?

```
SilentReporter s;
Reporter& r=s;
cout << r.report();
```

Answer to Q13(d): output and reason

14. Templates and stream reasoning. (14 points)

(a) Write a function template [4 points]

Write halfValue(T value), with one type parameter T and return type T. Its body must return value/2. Then state the types and values below [4 more points; 1 per line]. Do not change the template to force floating-point division.

```
auto a=halfValue(7);
auto b=halfValue(7.0);
double c=halfValue(7);
auto d=halfValue<double>(7);
```

Answer to Q14(a): function template

Types and values (4 points)

Variable	Type	Value
a		
b		
c		
d		

14. Templates and stream reasoning (continued)

(b) Exact stream output [4 points]

```
istringstream in("24\nAda Lovelace\n");
int id;
string first, name;
in >> id;
getline(in,first);
getline(in,name);
ostringstream out;
out << fixed << setprecision(2) << setfill('_');
out << setw(6) << 3.5 << '|' << 2.0;
cout << id << ":[" << first << "]\n";
cout << '[' << name << "]\n";
cout << out.str() << '\n';
```

Write all three output lines. Explain the first getline result and which formatting settings persist.

Answer to Q14(b): three output lines and explanation

(c) Recover after failed input [2 points]

```
istringstream numbers("10 20 x 30");
int n, total=0;
while(numbers >> n) total+=n;
```

State total after the loop. Write code that clears the failure, discards the single bad character x, then reads and adds the remaining 30. Explain why clear() alone is insufficient.

Answer to Q14(c): total, recovery code and explanation